BÁO CÁO TIẾN ĐỘ LUẬN VĂN TỐT NGHIỆP

Đề tài: Mở rộng EvoMaster bằng một oracle metamorphic phát hiện lỗi phân quyền có trạng thái theo thời gian trong REST API

Ngày báo cáo: 23/08/2026

Trạng thái tổng quát: Phần hiện thực và phần thực nghiệm **đã hoàn tất và đã đo xong**. Công việc còn lại chủ yếu là **viết luận văn** (tiếng Việt) và rà soát lần cuối phần công trình liên quan.

---

0. Tóm tắt một trang (đọc phần này là đủ nắm)

Hạng mục	Kết quả
Đóng góp	Thêm vào EvoMaster khả năng phát hiện lớp lỗi *"quyền đã bị thu hồi nhưng token cũ vẫn dùng được"* — lớp lỗi mà **chưa fuzzer REST API nào** phát hiện được
Mã nguồn	**+541 dòng Kotlin trên 4 file** trong `core` của EvoMaster, chỉ thêm chứ không sửa; **không đụng vào lõi thuật toán tiến hoá (MIO)**
Mã lỗi mới	`970 — SECURITY_STATEFUL_AUTHORIZATION`
Hệ thống thử nghiệm	3 hệ thống, mỗi hệ thống có 2 phiên bản (lỗi / đã vá) để so sánh vi sai
Số lần chạy đã đo	**230 lần chạy** có ghi log đầy đủ (Tier-1: 80, Tier-2: 120, Tier-2b: 30)
Độ chính xác	**0 báo động giả trên toàn bộ 80 lần chạy vào phiên bản đã vá** của hai hệ thống chính (+ 5 lần chạy Tier-2b)
Độ bao phủ (recall)	Tier-1: **80%**; Tier-2: **25%**; nguyên nhân mất recall đã được **chứng minh nhân quả** bằng một thí nghiệm can thiệp (đưa lên 100% và 65%)
Kết quả âm quan trọng	Trên một **lỗi thật của bên thứ ba có CVE** (pyLoad, CVE-2026-41133), công cụ **không tự dựng được** kịch bản — và đã chỉ ra chính xác 3 nguyên nhân trong mã nguồn
Khoảng trống nghiên cứu	Vẫn còn mở (kiểm tra trực tiếp lần cuối 12/08/2026)
Rủi ro lớn nhất còn lại	Thời gian viết luận văn; và dữ liệu đo hiện **chưa được quản lý phiên bản bằng git**

Điểm cần nhấn khi trình bày: con số đáng giá nhất của luận văn **không phải là recall**, mà là **0 báo động giả**, cộng với việc **toàn bộ phần recall bị mất đều nằm ở khâu dựng kịch bản, không nằm ở khâu phán quyết** — và điều này đã được chứng minh bằng thí nghiệm, không phải bằng lập luận.

---

1. Bài toán: lỗi cần bắt là lỗi gì

Kịch bản, đúng như nó xảy ra trong hệ thống thật:

1. alice là **ADMIN**. Cô đăng nhập, nhận một token, và xoá được báo cáo. Bình thường.
2. Một admin khác **hạ quyền alice xuống VIEWER**. Về nguyên tắc, từ giây này alice không được xoá gì nữa.
3. alice dùng lại **đúng token cũ** đó — và **vẫn xoá được**.

Nguyên nhân gốc rất phổ biến: hệ thống tin vào `role` đã được ghi sẵn *bên trong JWT* lúc đăng nhập, thay vì đọc lại quyền hiện tại của người dùng từ cơ sở dữ liệu. Token cũ mang theo quyền ADMIN đã lỗi thời, và nó còn hiệu lực cho tới khi hết hạn.

Đây không phải lỗi hàn lâm. Phân quyền là **lớp lỗ hổng API số 1 trong thực tế** (OWASP API Security Top 10). Biến thể có trạng thái nêu trên là biến thể nguy hiểm nhất, vì nó xảy ra đúng vào lúc tổ chức *tưởng* mình đã thu hồi quyền xong. Một bằng chứng cụ thể: đúng cơ chế này tồn tại trong **pyLoad** (CVE-2026-41133, CWE-613, CVSS 8.8) — và như mục 7.4 trình bày, **bản vá được công bố cho lỗi đó thực tế chưa hề vá**.

Luận văn giới hạn phạm vi vào biến thể **A1 — hạ vai trò (role downgrade)**, nơi câu "phải bị từ chối" là rõ ràng không tranh cãi. Hai biến thể cùng họ (A2 — chuyển quyền sở hữu; A3 — hết hạn gói dịch vụ) được **định nghĩa hình thức** để chứng minh lược đồ tổng quát hoá được, nhưng chỉ hiện thực nếu còn thời gian.

---

2. Khoảng trống nghiên cứu — vì sao chưa ai làm

2.1 Công trình đối chuẩn trực tiếp

Şahin, Zhang & Arcuri (2026), arXiv:2604.00702 — chính nhóm tác giả của EvoMaster. Họ vừa thêm **9 oracle bảo mật** vào công cụ (mã lỗi F200–F210), trong đó F204–F208 là về phân quyền.

Điểm mấu chốt: **cả 9 oracle đó đều so sánh giữa những người dùng khác nhau, tại một thời điểm đóng băng.** Chúng gửi cùng một yêu cầu bằng hai tài khoản khác quyền rồi so kết quả. Không oracle nào mô hình hoá một **phép chuyển đổi làm giảm quyền diễn ra theo thời gian**.

Hai trục **vuông góc với nhau**, không cạnh tranh:

	Trục so sánh	**Câu hỏi được trả lời**
Şahin và cộng sự (đối chuẩn)	khác người, **cùng một thời điểm**	ai đang được phép?
Luận văn này	**cùng một người, khác thời điểm**	quyền đã thu hồi có thật sự mất hiệu lực?

> **Lưu ý về tính trung thực học thuật:** phần "future work" của bài báo đối chuẩn chỉ nói chung chung (*"Future work will aim at designing new automated oracles to identify other kinds of security faults in REST APIs…"* — đã kiểm chứng nguyên văn từ bản gốc). Họ **không** nêu tên hướng stateful/temporal. Vì vậy khoảng trống trong luận văn phải được lập luận **theo cấu trúc** (mô hình của họ không có trục thời gian), **tuyệt đối không** được viết là "chính tác giả đã chỉ ra hướng này".

Một chi tiết củng cố thêm: mục §5.4 của chính bài báo đó coi các endpoint làm thay đổi vai trò/thông tin đăng nhập là **phiền toái** (vì chúng làm hỏng credential đang dùng) — trong khi đó chính là thứ luận văn này biến thành một phép kiểm thử.

2.2 Các công trình lân cận (đã kiểm tra trực tiếp, lần cuối 12/08/2026)

Không công trình nào — học thuật hay thương mại — chiếm đúng vị trí này. Mỗi láng giềng đều lệch trên một trục **nêu tên được**:

Công trình	Lệch ở đâu
BACScan (CCS 2025, doi:10.1145/3719027.3744825) — đối thủ mạnh nhất về access control	Khác-người / một-thời-điểm; nhắm **web app**, không phải REST/OpenAPI
BACFuzz (arXiv:2507.15984)	Grey-box cho **PHP web app**; BOLA/BFLA, khác-người, không có phép chuyển đổi
GraphQLer (arXiv:2504.13358)	Nhắm **GraphQL**, không phải REST. Đáng chú ý: bài này **tự tay dựng chuỗi 3 bước** `CREATE → DELETE → READ`, tức khả năng dựng chuỗi nằm trong tầm tay họ — **nhưng không chuỗi nào đổi vai trò rồi phát lại credential cũ**. Đó chính xác là delta của luận văn
ARMeta (arXiv:2605.28321) — gần nhất về **phương pháp** (metamorphic + REST)	Các quan hệ thuần **chức năng**; phần nghiên cứu tình huống RBAC **không định nghĩa quan hệ phân quyền nào**
WuppieFuzz (arXiv:2512.15554) — fuzzer REST có trạng thái	**Không có oracle phân quyền nào cả**; tính "có trạng thái" chỉ là phụ phẩm
AuthREST (arXiv:2509.10320)	Nhắm **xác thực**, không phải phân quyền
BacAlarm (arXiv:2512.19997)	Là **bộ phát hiện bất thường lúc chạy** (ML/IDS), không phải oracle kiểm thử
Công cụ thương mại (StackHawk, Invicti, Equixly, BOLABuster)	Đều là ma trận so sánh chéo người dùng tại một thời điểm

Còn một nguồn chưa kiểm chứng được trực tiếp: **Spec2SeqFuzz** (Electronics 15(11):2309) — trang MDPI trả về 403 nên **mới chỉ đọc được phần tóm tắt**; khi trích dẫn bắt buộc phải ghi rõ điều này.

---

3. Oracle — định nghĩa chính xác

Đây là phần lý thuyết của luận văn. Đặc tả đầy đủ nằm ở `docs/metamorphic-authz-relations.md`.

3.1 Quan hệ đúng là "thực thi", không phải "đơn điệu"

Cách nói trực giác là *"gỡ bỏ một quyền thì không bao giờ được làm tăng khả năng truy cập"* — nhưng nói chặt chẽ thì đó là **tính đơn điệu (monotonicity)**, và **lỗi A1 KHÔNG vi phạm tính đơn điệu**: alice xoá được trước khi bị hạ quyền, và vẫn xoá được sau đó — khả năng truy cập của cô **không hề tăng**.

Cái bị vi phạm là **tính thực thi (enforcement)**: một quyền đã bị gỡ vẫn còn được **honor** thông qua một **credential đã cấp phát từ trước (token cũ)**. Quan hệ metamorphic đúng là một phép kiểm tra **thu hồi có điều kiện canh gác**:

> Một hành động **đòi hỏi đúng cái quyền vừa bị gỡ** (gọi là *guard*), một khi đã được quan sát là **ĐƯỢC PHÉP**, thì sau khi quyền bị gỡ phải trở thành **BỊ TỪ CHỐI** — khi **dùng lại đúng credential đó**.

Về "guard" là bắt buộc, không phải trang trí. Nếu bỏ nó, quan hệ trở thành sai: `GET /reports` vốn cho phép cả VIEWER, nên nó được phép **cả trước lẫn sau** khi hạ quyền (không có lỗi) — một luật không guard sẽ báo động giả ngay tại đây. Đây **không phải lo xa lý thuyết**: một phiên bản đầu của luật phán quyết đã bỏ guard và **đã thực sự sinh ra đúng báo động giả đó** trên VAmPI (xem mục 8).

3.2 Công thức ba bước `cấp quyền → chuyển đổi → thử lại`

```
1. reset                              # trạng thái sạch, đã biết
2. đăng nhập p        → lấy c         # credential giữ nguyên suốt kịch bản
3. thực hiện a bằng c → kỳ vọng ĐƯỢC PHÉP    [nguồn: hành động canh gác chạy được khi còn
quyền]
4. áp dụng T (bởi người vẫn còn quyền) → làm giảm quyền của p   [phép chuyển đổi]
5. thực hiện a bằng c MỘT LẦN NỮA → quan sát  [dùng LẠI c, KHÔNG đăng nhập lại]
6. phán quyết:  ĐƯỢC PHÉP(3) VÀ BỊ TỪ CHỐI(5)  → ĐẠT 🟢
                ĐƯỢC PHÉP(3) VÀ ĐƯỢC PHÉP(5)   → LỖI 🔴
```

Mấu chốt nằm ở bước 5: phát lại token cấp trước khi hạ quyền. Nếu đăng nhập lại sau khi hạ quyền, ta nhận token VIEWER và **lỗi bị che mất** — đây chính là nước đi mà công trình đối chuẩn không bao giờ thực hiện. Bước 1–5 là **dựng kịch bản (construction)**; bước 6 là **phán quyết (judgment)**.

3.3 Các định nghĩa mà mã nguồn phụ thuộc vào

- **ĐƯỢC PHÉP = HTTP 2xx. BỊ TỪ CHỐI = HTTP 403. Không** coi 404 là từ chối (nhập nhằng). Ta dựa trên việc hệ thống kiểm tra **phân quyền trước khi kiểm tra sự tồn tại** của tài nguyên — đã kiểm chứng trực tiếp trên cả ba hệ thống thử nghiệm.
- **401 = KHÔNG KẾT LUẬN ĐƯỢC, không bao giờ là một phán quyết.** Token hết hạn giữa chừng ⇒ huỷ kịch bản và thử lại. Cả ba hệ thống đều đặt TTL 1 giờ để token cũ sống sót qua kịch bản (TTL mặc định 60 giây của VAmPI từng làm hỏng phép đo).
- **Điều kiện hợp lệ** (vi phạm bất kỳ điều nào ⇒ kịch bản **VÔ HIỆU**, *không* phải một kết quả): token còn hiệu lực · thứ tự 403-trước-404 đúng · **phép chuyển đổi thực sự đã có hiệu lực** · hành động nguồn thực sự bị chặn bởi quyền đã gỡ.
- **Nhận diện phép chuyển đổi T:** suy luận từ đặc tả OpenAPI + cách đặt tên endpoint (từ khoá `role/permission/member/subscription/plan/owner/grant/access`, động từ `PUT/PATCH/DELETE` trên tài nguyên người dùng/vai trò, trường body kiểu `role/plan/owner/active`).

---

4. Hiện thực trong EvoMaster

4.1 Quy mô thay đổi mã nguồn

Chỉ số	Giá trị
Số dòng thêm vào	**+541, thuần thêm mới, 0 dòng bị xoá**
Số file chạm vào	**4**
Lõi thuật toán tiến hoá (MIO)	**KHÔNG chạm tới**
Nhánh git	`stateful-authz-oracle`, 4 commit trên tag `v6.1.1`

```
EMConfig.kt                    +  9   (một cờ cấu hình tuỳ chọn cho thí nghiệm ablation)
ExperimentalFaultCategory.kt   +  6   (một entry enum: mã lỗi 970)
RestSecurityOracle.kt          +104   (phán quyết — mô phỏng theo handler F206 sẵn có)
RestSecurityBuilder.kt         +422   (dựng kịch bản + các hàm suy luận)
```

Đây là con số quan trọng khi trình bày: đóng góp **không phải** là viết lại một công cụ, mà là chèn một oracle mới vào đúng điểm mở rộng mà kiến trúc EvoMaster đã có sẵn.

4.2 Chèn vào đâu trong kiến trúc

- **Điểm móc vòng đời:** `Main.kt` → `phaseSecurity()` chạy **SAU** khi tìm kiếm kết thúc. Đây là điểm giảm rủi ro lớn nhất của cả đề tài: oracle là một **pha hậu xử lý**, không phải phẫu thuật vào lõi thuật toán tiến hoá.
- **Dựng kịch bản:** `RestSecurityBuilder.addForStatefulAccessControl()` + `buildStatefulAuthzTriple()`, cắm ngay sau `expandWithForbidden()` sẵn có (thừa hưởng luôn bằng chứng 403 và ngân sách thời gian của pha).
- **Phán quyết:** `RestSecurityOracle.handleStatefulAuthorization()`, mô phỏng theo `handleForbiddenOperation` (handler của F206 — oracle họ hàng gần nhất). ⚠️ Handler này chạy trên **MỌI** cá thể trong lời giải, không chỉ những kịch bản do builder tạo — đây là lý do các phép canh gác phải nằm trong handler.
- **Phát lại token cũ đến MIỄN PHÍ (phát hiện có tính quyết định):** trong EvoMaster, thông tin xác thực là một **"công thức đăng nhập" bất biến**, và mỗi test **đăng nhập một lần duy nhất, ngay đầu test**, rồi mọi hành động dùng chung bản đồ `tên người dùng → token`. Vì vậy chuỗi `[alice DELETE, superadmin hạ quyền alice, alice DELETE]` sẽ đăng nhập alice **một lần khi cô còn là ADMIN** và dùng token ADMIN đó cho **cả hai** lần DELETE. Nghĩa là **cơ chế phát lại token cũ đã có sẵn trong nền tảng**; phần việc của chúng ta chỉ là đặt đúng `auth.name` cho từng hành động.

4.3 Cách chạy

```
# tái lập đối chuẩn — dùng jar phát hành v6.1.1, không cần build
infra/em-venv/bin/evomaster --blackBox true --bbSwaggerUrl <spec> --bbTargetUrl <url> ...

# chạy oracle của luận văn — dùng jar build từ nguồn, thêm một cờ
  --useExperimentalOracles true --extraPhaseBudgetPercentage 0.6
```

Mã 970 là **thực nghiệm ⇒ mặc định TẮT**, nên nó không bao giờ làm nhiễu các lần chạy đối chuẩn.

---

5. Hệ thống thử nghiệm

Mọi hệ thống đều có **hai phiên bản** — *có lỗi* và *đã vá* — khác nhau đúng **một công tắc**. Nhờ vậy mọi khẳng định đều là một phép **so sánh vi sai**: công cụ phải **kêu trên bản lỗi và im trên bản đã vá**.

	Tier-1	**Tier-2**	**Tier-2b**
Hệ thống	`reports-api` — **do ta tự viết**	`vampi-seeded` — fork của VAmPI	**pyLoad** — ứng dụng thật, bên thứ ba
Công nghệ	Spring Boot 3 / Java 17	Python / Flask	Python / Flask
Vai trò trong đánh giá	Chân lý nền có kiểm soát	Tổng quát hoá sang app ta không viết	**Lỗi thật, không do ta cấy**
Nguồn gốc lỗi	Ta cấy	Ta cấy	**CVE-2026-41133** (bên thứ ba)
Cổng lỗi / đã vá	8080 / 8081	5002 / 5003	8090 / 8091
Chân lý nền (nguồn/chuyển đổi/thử lại)	`204/200/204` vs `204/200/403`	`200/200/200` vs `200/200/403`	`200/200/200` vs `200/200/403`

Vì sao Tier-2 quan trọng: các endpoint được đặt tên **theo đúng lối đặt tên của VAmPI**, cố ý **không** sao chép từ `reports-api`. Đó là điều làm cho Tier-2 trở thành phép thử thật sự của khả năng **suy luận phép chuyển đổi**, chứ không phải một lần lặp lại Tier-1.

Một phát hiện phụ đáng viết vào luận văn: trong khi cấy lỗi vào VAmPI, đã phát hiện và sửa một **lỗi có sẵn của VAmPI gốc** — hàm `register_user` đặt tên sách bằng `randrange(100)` trên một cột UNIQUE, khiến `GET /createdb` **500 ngẫu nhiên**, đủ thường xuyên để làm hỏng một lô 20 lần chạy.

---

6. Kết quả chính (Tier-1 và Tier-2)

Thiết lập: 20 lần chạy mỗi nhánh, seed 1–20, cấu hình được giữ **giống hệt nhau** giữa các nhánh, reset dữ liệu trước mỗi lần chạy. Mỗi lần chạy ghi lại **bộ ba mã trạng thái thực tế** `[nguồn / chuyển đổi / thử lại]`, nhờ đó **khâu dựng kịch bản và khâu phán quyết được đếm tách rời** — đây là sự phân biệt mà toàn bộ bảng dưới đây dựa vào.

Hệ thống (bản lỗi)	Dựng được kịch bản hợp lệ	Phát hiện đầu-cuối	Báo động giả trên bản đã vá	Đối chuẩn Şahin
Tier-1 `reports-api`	16/20	**16/20 = 80%** [KTC 95%: 58–92]	**0/20** (chặn trên 16%)	—
Tier-2 `vampi-seeded`	5/20	**5/20 = 25%** [KTC 95%: 11–47]	**0/20** (chặn trên 16%)	**0/20**, 0/20 kịch bản

Tier-2 là **ma trận đầy đủ 4 nhánh** (công cụ của ta × đối chuẩn) × (bản lỗi × bản đã vá) = 80 dòng dữ liệu.

6.1 Bốn ô TP / TN / FP / FN (trả lời trực tiếp góp ý của thầy)

Đây là góp ý thầy đã nêu ở lần đánh giá trước, nên báo cáo đủ cả bốn ô — **và trên hai cơ sở tính khác nhau**, vì sự khác biệt giữa chúng chính là chỗ dễ bị hiểu sai nhất:

Cơ sở A — tính trên mọi lần chạy (nghiêm khắc nhất, đây là con số chính thức):

	Tier-1	Tier-2
TP (bản lỗi, có báo)	16	5
FN (bản lỗi, không báo)	**4**	**15**
TN (bản đã vá, không báo)	20	20
FP (bản đã vá, có báo)	**0**	**0**

Cơ sở B — chỉ tính những lần chạy thực sự dựng được kịch bản hợp lệ:

	Tier-1	Tier-2
TP / FN	16 / **0**	5 / **0**
TN / FP	16 / **0**	5 / **0**

> ⚠️ **Đây là chỗ phải trung thực và phải tự nói ra trước khi người phản biện chỉ ra.** Trên Cơ sở B, tỉ lệ "phán quyết đúng 100%" **gần như mang tính định nghĩa**, không phải một kết quả độc lập: một kịch bản được coi là *hợp lệ* một phần dựa trên mã trạng thái của bước thử lại, mà luật phán quyết cũng chính là dựa trên mã trạng thái đó. Điều mà Cơ sở B **thực sự** chứng minh thì hẹp hơn: handler **tự suy lại** bộ ba từ cá thể (cùng credential, đúng thứ tự, đúng vị từ chuyển đổi) và **chưa bao giờ bất đồng với builder**; và lỗi đã cấy biểu hiện **tất định**, không chập chờn.
>
> Ngược lại, **ô FP = 0 là phép đo độc lập thật sự**, vì handler quét **mọi** cá thể trong lời giải chứ không chỉ những kịch bản do builder dựng — nó có thừa cơ hội để kêu bậy và đã không kêu. Cơ hội đó không phải giả định: **trước khi thêm vế guard ở mục 3.1, nó ĐÃ từng kêu** trên những test hoàn toàn không có phép chuyển đổi nào.

"Vô hiệu" là điều kiện tiên quyết, không phải một phán quyết. Một kịch bản VÔ HIỆU không phải bằng chứng theo hướng nào cả — đếm nó thành true negative sẽ **thổi phồng** độ đặc hiệu. Vì vậy: **toàn bộ phần recall bị mất nằm ở khâu dựng kịch bản và thực thi, không có phần nào nằm ở khâu phán quyết.**

6.2 Đối chuẩn mù trên hai trục độc lập (cả hai đều được đo, không phải giả định)

Đây là lập luận chứng minh **cả hai** đóng góp của luận văn (bộ sinh kịch bản **và** oracle) đều cần thiết, chứ không phải chỉ một:

1. **Không thể PHÁN QUYẾT** — các oracle F204–F208 đều là một-thời-điểm/so-chéo-người; không cái nào mô hình hoá "quyền đã giảm ⇒ phải bị từ chối", nên một mã 204 sau khi hạ quyền bị đọc thành **thành công**.
2. **Không thể DỰNG được kịch bản** — trong các test do đối chuẩn sinh ra ở Tier-1: 5 test có chạm `PUT /users/{u}/role`, 32 test có sửa `/reports`, nhưng **0 test làm cả hai**. Kiểm chứng độc lập trên Tier-2: **0/20** lần chạy của đối chuẩn dựng được kịch bản A1, ngay cả khi một lệnh ghi vai trò và một lệnh ghi có canh gác cùng xuất hiện trong một test. Phép đo này (`baseline_blindness.py`) là **bắt buộc**, vì nhánh đối chuẩn chạy với mã 970 bị tắt — nếu chỉ nói "đối chuẩn được 0 điểm" thì lập luận sẽ **vòng quanh**.

Bằng chứng đối chuẩn đã được cấu hình ĐÚNG: nó vẫn phát hiện lỗi F209 trên **17/20** lần chạy. Con số 0 của nó ở lỗi có trạng thái là **mù về cấu trúc**, không phải cấu hình sai.

---

7. Ba thí nghiệm bổ sung

7.1 Thí nghiệm ablation: chứng minh nhân quả nguyên nhân mất recall

Mục 6 cho thấy recall thấp. Câu hỏi tự nhiên của người phản biện là *"vì sao?"*. Câu trả lời **không** được phép là suy đoán, nên đã làm một thí nghiệm **can thiệp đúng một biến**.

Giả thuyết: *chính quá trình tìm kiếm của fuzzer đã phá huỷ tiền đề của phép kiểm thử thời gian của chính nó* — trong lúc fuzz, công cụ **hạ quyền và xoá sạch chính những tài khoản** mà kịch bản cần.

Can thiệp: thêm cờ tuỳ chọn `--statefulAuthzResetEndpoint`, **mặc định TẮT**, gọi endpoint reset của chính hệ thống ngay trước mỗi lần thử dựng kịch bản. Mọi thứ khác giữ nguyên hoàn toàn, nên các dòng ghép cặp 1–1 với mục 6. **80 lần chạy.**

Nhánh	Dựng được kịch bản	Phát hiện	Recall [KTC 95%]	Báo động giả
Tier-1 bản lỗi, **không** reset	16/20	16	80% [58–92]	0/20
Tier-1 bản lỗi, CÓ reset	**20/20**	**20**	**100% [84–100]**	**0/20**
Tier-2 bản lỗi, **không** reset	5/20	5	25% [11–47]	0/20
Tier-2 bản lỗi, CÓ reset	**13/20**	**13**	**65% [43–82]**	**0/20**

Bốn điều thí nghiệm này thiết lập, theo thứ tự quan trọng:

1. **Chẩn đoán là nhân quả, không chỉ là "phù hợp".** Loại bỏ sự phá huỷ trạng thái thì recall mất đi cũng biến mất.
2. **100% recall mất nằm ở dựng kịch bản, 0% nằm ở phán quyết** — recall dịch 80→100% và 25→65% trong khi **luật phán quyết không hề thay đổi một dòng**.
3. **Recall không phải đánh đổi bằng độ chính xác.** Reset làm cho bản **đã vá** dựng được **nhiều** kịch bản hơn (16→20 và 5→13), tức là có **nhiều cơ hội kêu bậy hơn**, mà vẫn **không kêu bậy lần nào**.
4. **Reset không làm nhiễu các oracle khác** — đây là biến gây nhiễu cần chặn trước, và đã được **đo**, không phải giả định. Tier-1: số lượng F100/F101/F205 **giống hệt nhau** giữa hai chế độ. Tier-2 lệch nhẹ (F205 4→2, F209 18→19) và **lệch giống nhau ở cả bản lỗi lẫn bản đã vá**, tức là hiệu ứng tất định theo seed chứ không phải nhiễu. **Phải báo cáo độ lệch này, không được giấu.**

Một kết quả nghịch — và là kết quả giàu thông tin nhất ở đây. Seed 15 phát hiện được khi **không** reset, và **mất** khi có reset. Nguyên nhân đã **kiểm chứng bằng cách đọc test sinh ra**: quá trình tìm kiếm đã vô tình **THĂNG quyền** nạn nhân `name1` lên admin (endpoint role chấp nhận cả giá trị `"admin"`). Reset trả `name1` về người dùng thường, hành động nguồn thành 403, kịch bản trở nên vô hiệu. Vậy: **sự phá hoại trạng thái không thuần tuý là phá hoại — đôi khi nó vô tình TẠO RA tiền đề**. Nó cũng lộ ra một hạn chế thiết kế tinh vi hơn: **ứng viên được chọn dựa trên những gì quá trình tìm kiếm đã quan sát, nhưng kịch bản lại chạy trên trạng thái vừa reset — hai thứ này không khớp nhau.** Cách sửa (xác thực lại bằng chứng ứng viên sau khi reset) là hướng phát triển tương lai.

Cách trình bày đã chốt: con số KHÔNG reset là con số chính thức — đó là những gì black-box thuần tuý làm được với một ứng dụng không có endpoint reset. Nhánh có reset là một **thí nghiệm giải thích**, **không phải** một cấu hình tốt hơn. Cờ này xuất xưởng ở trạng thái TẮT. Câu hỏi phản biện dự kiến — *"vậy là em reset server cho công cụ của em đẹp số à?"* — được trả lời bằng đúng cách đóng khung đó, cộng với việc **các nhánh đo báo động giả cũng được reset y hệt**.

7.2 Bằng chứng "vế guard là bắt buộc"

Một phiên bản đầu của luật phán quyết chấp nhận **bất kỳ** lệnh gọi thành công nào của người dùng khác là "phép chuyển đổi", tin rằng khâu dựng kịch bản đã đảm bảo điều đó. **Đảm bảo ấy không đứng vững**, vì handler chạy trên **mọi** cá thể. Trên VAmPI nó đã kêu 970 trên những test **không có phép chuyển đổi nào cả** (ví dụ `GET /me` → một `GET /_debug` không liên quan của người khác → `GET /me`).

Luật hiện tại yêu cầu hành động xen giữa phải **trông giống một phép chuyển đổi làm giảm quyền** (dùng chung một vị từ với khâu dựng kịch bản, để phán quyết và dựng kịch bản **không thể trôi lệch nhau**) **và** hành động nguồn phải là một lệnh **ghi** — một lệnh GET vẫn đọc được thì không chứng minh được gì về một quyền **ghi** đã bị gỡ. **Chi phí: không mất true positive nào trên cả hai hệ thống.**

Đây nên vào luận văn như **bằng chứng cho thiết kế**, không phải như một mục sửa lỗi.

7.3 Tier-2b — pyLoad: một lỗi thật của bên thứ ba (trả lời góp ý thứ hai của thầy)

Góp ý thứ hai của thầy: *đừng chỉ dựa vào kịch bản tự cấy*. Tier-2b là câu trả lời, và nó trả lời được **đến đâu thì nói đến đó**.

Mục tiêu: pyLoad — một trình quản lý tải xuống được dùng rộng rãi, giao diện web Python/Flask. **CVE-2026-41133, CWE-613, CVSS 8.8**, báo cáo tháng 4/2026.

Nguyên nhân gốc **cùng dạng với cả hai hệ thống cấy lỗi** — bản thân điều này đã là một phát hiện có giá trị, vì nó chứng tỏ hai hệ thống cấy lỗi được thiết kế quanh một cơ chế **thật sự xảy ra ngoài đời**: `set_session()` chép `role`/`permission` vào session lúc đăng nhập; phân quyền về sau đọc **bản chép đó**; `set_user_permission()` ghi vai trò mới vào cơ sở dữ liệu và **không vô hiệu hoá session nào**.

> **Phát hiện phụ đáng một câu trong luận văn: thượng nguồn chưa hề vá lỗi này.** Advisory tuyên bố đã vá ở `0.5.0b3.dev98`, nhưng hàm `is_authenticated` **giống hệt từng byte** qua `dev97`, `dev98`, `dev99`, `dev100`, `dev101` (mới nhất trên PyPI) **và** nhánh `develop`. Một lớp lỗi được tuyên bố đã vá trong khi thực tế chưa — đó chính là luận cứ cho việc cần một **oracle tự động** thay vì rà tay.

(a) Kết quả trước khi mở rộng: công cụ KHÔNG tìm ra — và đây là kết quả có giá trị

Đo bằng **đúng bản build đã sinh ra mọi con số của Tier-1 và Tier-2**, trước khi có bất kỳ thay đổi nào để phục vụ pyLoad. Thứ tự đó là điểm mấu chốt: nhánh này ghi lại công cụ làm gì trên một **lỗi thật mà nó chưa từng được điều chỉnh cho phù hợp**.

Nhánh	Số lần chạy	Mã 970	Tìm ra phép chuyển đổi	Độ bao phủ endpoint
`pyload-vuln`	5	**0/5**	0/5	78–83%
`pyload-fixed`	5	**0/5**	0/5	65–83%

Các endpoint ĐÃ được chạm tới; chúng chỉ chưa bao giờ được ghép lại. Đếm theo từng test: mỗi lần chạy có **10** test gọi guard, **10** test gọi phép chuyển đổi, và **0** test gọi cả hai. Không phải lỗi về khả năng tiếp cận — mà bộ sinh **không bao giờ đặt chúng vào cùng một chuỗi với credential được phát lại**.

Đọc mã nguồn, có **ba nguyên nhân độc lập, mỗi cái đủ để giết chết kịch bản**:

1. **Từ vựng chuyển đổi.** Endpoint của pyLoad là `POST /json/update_users` — **không khớp cái nào** trong 8 từ khoá đang dùng.
2. **Thành ngữ chuyển đổi.** pyLoad dùng lối **"trạng thái mong muốn dạng khối"**: body là `{"update_data": {...}}` mà **khoá** mang tên nạn nhân, và hạ quyền được biểu đạt bằng **sự vắng mặt của một khoá**. Không có trường user, không có trường role để bám vào. **Nguyên nhân này KHÔNG thể khắc phục bằng cách mở rộng danh sách từ khoá.**
3. **Động từ của hành động nguồn.** Khâu dựng kịch bản chỉ xét `PUT`/`PATCH`; bề mặt `/json/*` của pyLoad là **RPC trên POST và GET** — không có lấy một PUT/PATCH/DELETE nào trong 23 thao tác.

> ⚠️ **Không được viết** "oracle đã bắt được nếu khâu dựng kịch bản làm được việc". Việc kiểm chứng bằng tay ở trên chỉ chứng minh **hệ thống pyLoad có biểu hiện quan hệ đó** — một sự thật về pyLoad, không phải về handler của ta. Với bản build đã đóng băng, **khâu phán quyết cũng bị chặn** (nguyên nhân 1 chặn cả hai nửa).
>
> ⚠️ **Trung thực về cỡ mẫu:** chặn trên Wilson 95% cho 0/5 là **43%**. Năm seed **không đủ** để nói "không bao giờ phát hiện". Khẳng định dựa vào **nguyên nhân cấu trúc đọc từ mã nguồn** (log của builder rỗng hoàn toàn xác nhận trực tiếp); các seed chỉ cho thấy nguyên nhân **không phụ thuộc seed**.

Một quan sát ngoài dự kiến, làm sắc thêm hạn chế ở mục 8: sau lô chạy, kiểm tra cơ sở dữ liệu thấy **cả `alice` lẫn `superadmin` đều đã bị hạ xuống USER**, cùng khoảng năm mươi tài khoản rác do fuzzer tạo. Nguyên nhân nằm ở chính thành ngữ đó: `update_users` nhận trạng thái mong muốn của **toàn bộ người dùng cùng lúc**, nên **một lệnh gọi với body thiếu sót sẽ hạ quyền TẤT CẢ MỌI NGƯỜI**. Nghĩa là: mức độ hệ thống tự phá huỷ tiền đề của nó phụ thuộc vào **bán kính sát thương của endpoint chuyển đổi** — một thuộc tính của **ứng dụng**, không phải của fuzzer.

(b) Khi khai báo sẵn phép chuyển đổi: phần còn lại của quy trình có chạy đúng trên mã bên thứ ba

Ba nguyên nhân ở trên đều nằm trong **khâu xử lý phép chuyển đổi**. Vậy chưa thể biết phần còn lại (chọn nạn nhân, tìm guard, phát lại credential, cơ chế thử-lại-khi-vô-hiệu, phán quyết) có chạy được trên một ứng dụng ta không viết hay không. Cờ `--statefulAuthzTransition` cung cấp đúng thứ duy nhất không suy luận được, để đo phần còn lại. 5 seed × {lỗi, đã vá} × {có reset, không reset}:

Nhánh	Mã 970	Dựng được kịch bản	Số lần thử vô hiệu	Bộ ba trạng thái
bản lỗi, không reset	0/5	0	**138**	— (không chạy được lần nào)
bản lỗi, CÓ reset	**5/5**	5	0	`200/200/200`
bản đã vá, không reset	0/5	0	**138**	—
bản đã vá, CÓ reset	**0/5**	5	0	`200/200/403`

Được phép tuyên bố gì và không được phép tuyên bố gì:

- ❌ **Không được** trích một tỉ lệ phát hiện từ nhánh này. Phép chuyển đổi đã được khai báo sẵn, nên nó **không đo gì về recall black-box**. Con số recall trung thực của Tier-2b là **0/5** ở phần (a).
- ✅ **Được:** *với điều kiện* có phép chuyển đổi, **mọi khâu phía sau đều chạy đúng trên mã của bên thứ ba** — chọn nạn nhân và người thao tác, tìm guard, phát lại cùng credential, và luật phán quyết đều chạy **không sửa đổi** và cho phán quyết đúng trên **cả hai** phiên bản.
- ✅ **Được, và đây là con số độ chính xác duy nhất ở đây: 0 báo động giả trên 5 lần chạy bản đã vá mà mỗi lần đều dựng được một kịch bản thật.** (Nhánh không-reset **không** được tính là true negative vì chúng không dựng được kịch bản nào.) Với n=5, chặn trên Wilson 95% là 43% — nên câu đúng là *"không quan sát thấy báo động giả nào"*, **không phải** *"tỉ lệ báo động giả thấp"*.

Sự phá huỷ trạng thái ở đây khắc nghiệt hơn hẳn hai hệ thống kia: reset đưa phát hiện từ **0/5 lên 5/5** (Tier-1: 80→100%, Tier-2: 25→65%). Lý do là bán kính sát thương ở trên — không phải *thiếu* người thao tác còn quyền, mà là **không còn ai cả**: 138 lần thử trả về `200/401/200`, tức chính phép chuyển đổi bị từ chối.

Một ví dụ đã kiểm chứng bằng tay. Test `test_56_post_on_status_stillAllowedAfterPermissionRevoked` của seed 1: đăng nhập `pyload` và `alice`, gọi `POST /json/status` bằng `pyload` (200), để `alice` hạ quyền `pyload`, rồi lặp lại `POST /json/status` với **đúng cookie cũ của `pyload`** và nhận 200. Chạy tay đúng chuỗi đó trên bản đã vá cho **403** ở

bước cuối. Đáng chú ý: công cụ **không** đi theo kịch bản mà hệ thống này được thiết kế quanh nó — nó tự chọn `pyload` làm nạn nhân thay vì `alice`, và `/json/status` làm guard thay vì `/json/generate_apikey`. Đó là bằng chứng khâu dựng kịch bản **đang thực sự tự tìm kiếm**, chứ không phát lại một kịch bản đã được lập trình sẵn.

(c) Sáu điểm lệch chuẩn phải khai báo, và ranh giới "của ta / của họ"

Trung thực về mặt phương pháp: cần **6 điểm lệch** so với một lần chạy nguyên bản. **Không điểm nào tạo ra, che giấu hay dịch chuyển lỗi**, nhưng luận văn phải liệt kê đủ. Chúng **không cùng loại** — bốn điểm đầu chạm vào hệ thống hoặc đặc tả, hai điểm sau chỉ là tham số dòng lệnh của công cụ:

#	Ở đâu	Điểm lệch	Vì sao vô hại
1	Hệ thống, **cả hai** bản	Tắt CSRF	CSRF trực giao với phân quyền, và áp dụng cho **cả hai** bản nên **triệt tiêu trong phép so sánh vi sai**
2	Hệ thống, **chỉ bản vá**	Bản vá đối chứng là **do ta viết**	Thượng nguồn không có bản vá nào chạy được (xem trên). Bản vá không phải bịa: nó áp dụng đúng thao tác đọc lại cơ sở dữ liệu mà nhánh `X-API-Key` của chính pyLoad **đã** làm sẵn
3	**Đầu vào**, không phải hệ thống	Đặc tả OpenAPI cho `/json/*` viết tay	Đặc tả là **đầu vào** của black-box fuzzing, không phải một phần của hệ thống. Để tránh "mớm bài", đặc tả mô tả **cả 20 route / 23 thao tác**, không chỉ ba cái mà kịch bản cần
4	Hệ thống, **cả hai** bản	Chèn endpoint `/thesis/reset`	Chỉ ghi vào bảng `users`, không chứa logic phân quyền; có ở **cả hai** bản nên triệt tiêu trong phép so sánh
5	Tham số công cụ	`--endpointExclude "/api/kill,/api/restart"`	Là **tuỳ chọn có sẵn của EvoMaster**. Hai endpoint này **giết tiến trình hệ thống** trong phút đầu tiên. Chỉ loại đúng hai cái đó
6	Tham số công cụ, **chỉ nhánh (b)**	`--statefulAuthzTransition ...`	**Điểm lệch nặng nhất**, và luận văn phải nói rõ như vậy; nó **không** triệt tiêu, nên **không tỉ lệ phát hiện nào được trích từ nhánh này**

Ranh giới có thể kiểm chứng được, không phải lời hứa: ở bản **có lỗi**, hai file duy nhất bị sửa là `config.py` (dòng CSRF) và `json_blueprint.py` (route reset). File chứa **toàn bộ quyết định phân quyền** — `helpers.py` — là **mã thượng nguồn nguyên vẹn**: 0 dấu `THESIS` ở bản lỗi, đúng 2 ở bản vá (chính là bản vá đối chứng).

> **Kết luận Tier-2b, nói thẳng:** *Chưa có true positive trên một lỗi không do ta cấy* — điều đó vẫn còn để ngỏ, và mục (a) giải thích chính xác vì sao. Nhưng **giá trị của nó là một kết quả âm đáng giá hơn kết quả dương mà nó thay thế:** hạn chế cũ trong luận văn ("một endpoint hạ quyền có tên ngoài từ vựng sẽ bị bỏ sót") vốn chỉ là **lập luận từ việc đọc mã**. Giờ nó là **quan sát trên mã của bên thứ ba**, và bức tranh thật **lớn gấp ba lần** so với lập luận đó dự đoán: bộ sinh của ta đang ngầm giả định **thiết kế RESTful** (endpoint vai trò dạng tài nguyên, tham số có tên, ngữ nghĩa động từ), và một ứng dụng thật, được triển khai rộng rãi, **vi phạm cả ba giả định**.

---

8. Hạn chế đã biết (phải đưa vào luận văn)

1. **Quá trình tìm kiếm phá huỷ tiền đề của chính phép kiểm thử thời gian của nó** — hạn chế chi phối nhất. Black-box không có cơ chế reset giữa pha tìm kiếm và pha bảo mật, và fuzzer dùng pha tìm kiếm để **hạ quyền và xoá đúng những tài khoản mà kịch bản cần**. Hai cơ chế, đều đã đo: (a) nạn nhân đã bị hạ quyền sẵn ⇒ hành động nguồn trả 403, mà black-box **huỷ test ngay tại lệnh tạo tài nguyên thất bại đầu tiên**, nên phép chuyển đổi thậm chí không được gửi; (b) trên API có endpoint xoá người dùng, tài khoản bị **xoá thẳng**

— trong test sinh ra ở Tier-2, `DELETE /users/v1/{username}` xuất hiện **75 lần** với `name1`, 17 lần `name2`, 10 lần `admin`. Vì vậy khâu dựng kịch bản **thử lại trên nhiều cặp (nạn nhân, người thao tác) ứng viên** và coi kịch bản không thực hiện được là VÔ HIỆU. Đây **luôn là false negative, không bao giờ là false positive**.

2. **Suy luận phép chuyển đổi mới ở mức từ khoá + tham số** — mục 7.3 cho thấy hạn chế này **lớn hơn** dự đoán ban đầu ba lần.
3. **Ứng viên được chọn dựa trên trạng thái quan sát trong lúc tìm kiếm, nhưng kịch bản chạy trên trạng thái vừa reset** — hai thứ không khớp (bài học từ seed 15, mục 7.1).
4. **Chưa có true positive trên lỗi không do ta cấy** (mục 7.3).
5. **Một tạo tác kỹ thuật ở Tier-1:** springdoc công bố nhầm tham số nội bộ `caller` thành query param bắt buộc. ⚠️ Đây là điểm **đầu tiên** người phản biện sẽ chỉ vào. **Đã kiểm chứng lại trực tiếp ngày 16/08/2026: phân quyền 100% do JWT quyết định** — bob (VIEWER) kèm `caller={effectiveRole:ADMIN}` vẫn nhận **403**; không token kèm `caller=ADMIN` nhận **401**. Nó **không phải cửa hậu**, nhưng nó làm bẩn đặc tả và thổi phồng số F101. Cách sửa đã biết (`@Parameter(hidden=true)`) — ưu tiên thấp.
6. **A2/A3 mới ở mức định nghĩa hình thức**, chưa hiện thực.

---

9. Chất lượng và khả năng tái lập

Đây là phần thường bị hỏi trong buổi bảo vệ, nên đã chuẩn bị sẵn:

- **Điểm quay lui (rollback) cho mọi con số đã đo:** commit `769c16d141`; file jar tương ứng được giữ lại kèm `.sha256`. **Xác minh bằng NỘI DUNG, không bằng tên file hay thời gian sửa** (thời gian build của jar sớm hơn commit 9 tiếng nên mtime không chứng minh được gì) — kiểm bằng cách giải nén lớp `EMConfig` và đối chiếu tập cờ cấu hình bên trong.
- **Kiểm tra hồi quy tự động** trước khi bất kỳ jar mới nào được dùng để trích lại một con số cũ: **21 dòng** (hệ thống, nhánh, seed, chế độ reset) được chạy lại với **đúng tham số gốc**; kỳ vọng được **đọc tự động từ `runs.csv` gốc** chứ không gõ lại bằng tay. Nó bao phủ cả lần phát hiện được, lần từ chối đúng, **và** cả những lần **không dựng được kịch bản** — nên một thay đổi làm nới lỏng điều kiện tiền đề sẽ lộ ra dưới dạng một lần chạy trước đây "trắng" nay lại dựng được kịch bản. **Lần chạy gần nhất 17/08/2026: 21/21 dòng tái lập chính xác.** Trong đó **12 dòng là nhánh bản-đã-vá** — bao gồm toàn bộ 4 cấu hình có reset, tức là những nhánh dựng **nhiều** kịch bản nhất trên bản đã vá và do đó là **cơ hội báo động giả dày đặc nhất** trong toàn bộ đánh giá.
- **Kiểm tra bằng tay (hand-audit), 16/08/2026, Tier-1:** đọc trực tiếp các test được sinh ra và xác nhận: hành động nguồn và hành động thử lại mang **cùng một** credential `auth_alice`, **không có lần đăng nhập lại nào** ở giữa; phép chuyển đổi do `auth_superadmin` thực hiện và có assert `res.json()["role"] == "VIEWER"` — tức việc hạ quyền **chứng minh được là đã có hiệu lực**; và lần chạy trên **bản đã vá chứa các hành động giống hệt từng byte** (cùng id báo cáo, cùng người dùng), **chỉ khác duy nhất ở bước thử lại** — `403` thay vì `200` — và **không có mã 970** ở đâu cả.
- **Tính toàn vẹn:** không lần chạy nào phát mã 970 mà chưa dựng được kịch bản (một mã 970 "mồ côi" chính là kiểu hỏng trước khi có vế guard).
- ⚠️ **Rủi ro còn tồn đọng, cần xử lý:** thư mục luận văn **không phải một repo git**, nên toàn bộ file CSV kết quả đo **không có quản lý phiên bản**. Đây là **rủi ro đứng riêng lớn nhất** đối với dữ liệu thực nghiệm.

---

10. Công việc còn lại và kế hoạch

Hiện thực và đánh giá **đã xong**. Thứ tự ưu tiên:

#	Việc	Mức	Ghi chú
1	**Viết luận văn (tiếng Việt)**	**Trọng tâm hiện tại**	Vật liệu đã sẵn sàng: mục 3 (hình thức hoá), 6–7 (kết quả + ablation), 4 (hiện thực), 2 (công trình liên quan)
2	**Kiểm chứng lại khoảng trống nghiên cứu** trực tiếp trước khi chốt phần công trình liên quan	Bắt buộc	Lần kiểm cuối 12/08/2026. Trích nguyên văn hoặc ghi rõ là diễn giải — không có ngoại lệ
3	Mở rộng suy luận phép chuyển đổi (tên trường body, giá trị enum, mô tả OpenAPI)	Tuỳ chọn	Mục 7.3 chỉ ra rằng riêng mở rộng từ khoá là **không đủ**
4	Xác thực lại bằng chứng ứng viên sau khi reset (bài học seed 15)	Tuỳ chọn	
5	Sửa tạo tác `caller` của springdoc rồi chạy lại một lô đối chuẩn "sạch"	Ưu tiên thấp	

Bốn điểm bắt buộc phải xuất hiện trong luận văn (nếu thiếu, người phản biện sẽ tìm ra):

1. **Quan hệ là "thực thi", không phải "đơn điệu"** (mục 3.1);
2. **Đối chuẩn mù trên hai trục — đo được, không phải giả định** (mục 6.2);
3. **Đọc precision và recall một cách trung thực**, kèm cảnh báo ⚠️ ở mục 6.1 về việc tỉ lệ có điều kiện mang tính định nghĩa;
4. **Thí nghiệm ablation reset trạng thái** kèm cách đóng khung "con số không-reset là con số chính thức".

Đánh giá rủi ro trung thực: một luận văn **đạt yêu cầu** giờ gần như chắc chắn — phần đóng góp đã được xây xong và đã đo. Một luận văn **xuất sắc** (tìm được nhiều lỗi thật trên nhiều API thật) thì không chắc, và **không bắt buộc**. Rủi ro thật sự còn lại lớn nhất là **thời gian và việc phình phạm vi trong giai đoạn viết**.

Phụ lục — Nơi lưu dữ liệu và cách tái lập

Nội dung	Đường dẫn
Mã nguồn oracle	`/Users/minhnguyen/IdeaProjects/EvoMaster`, nhánh `stateful-authz-oracle` (⚠️ `origin` là repo thượng nguồn — **không bao giờ push**)
Bản sao di động của mã	`infra/evomaster-ext/`
Đặc tả hình thức của oracle	`docs/metamorphic-authz-relations.md`
Hệ thống Tier-1	`infra/reports-api/` (34 file Maven, có `Dockerfile`)
Hệ thống Tier-2	`infra/vampi-seeded/` (mọi dòng thêm đều đánh dấu `THESIS SEED`)
Hệ thống Tier-2b	`infra/pyload-cve/` + `verify_ground_truth.sh`
Kết quả Tier-1	`infra/em-results/tier1-trials/`, `tier1-reset-trials/`, `tier1-audit/`
Kết quả Tier-2	`infra/em-results/tier2-trials/` (+ `baseline_blindness.py`), `tier2-reset-trials/`
Kết quả Tier-2b	`infra/em-results/tier2b-trials/`, `tier2b-declared-trials/`
Kiểm tra hồi quy	`infra/em-results/regression-check/regression_check.sh`
Tài liệu chi tiết Tier-2 / Tier-2b	`docs/tier2-seeded-benchmark-plan.md`, `docs/tier2b-pyload-target.md`

Mỗi thư mục kết quả chứa `runs.csv` (dữ liệu thô, một dòng một lần chạy), `progress.log`, và script harness đã dùng để sinh ra chúng.